Jackett-in-the-Lava-Local-Stack — Deep Research Dossier

> Status: **research + recommendation only** (no code written, no stack files modified). Author: research agent, 2026-06-08. Scope: integrating **Jackett** as a containerized service inside the Lava local stack, fronted by `lava-api-go` over **Torznab**, orchestrated via the `vasic-digital/Containers` submodule (rootless podman/docker, docker-compose, `_lava-api._tcp` mDNS) — the same model `lava-api-go` already uses. Jackett does **not** ship inside the APK; it is a LAN/host-side container. Every external claim is cited with a URL. Items that could not be confirmed from primary sources are explicitly marked `UNCONFIRMED:` per HelixConstitution §11.4.6 (no-guessing vocabulary).

---

0. How this maps onto the existing Lava stack (grounding)

Verified from the working tree (`docker-compose.yml`, `lava-api-go/cmd/lava-api-go/main.go`):

- `lava-api-go` runs with `network_mode: host` "because mDNS requires host net", advertises itself on the LAN via mDNS service type **`_lava-api._tcp`** (`LAVA_API_MDNS_TYPE` default in `lava-api-go/internal/config/config.go:102`), listens on `:8443` (HTTP/3 + HTTP/2, TLS).
- Postgres + observability containers use the `lava-net` bridge; only `lava-api-go` is host-net.
- The Android debug build auto-discovers `lava-api-go` via `_lava-api._tcp`. The app talks **only** to `lava-api-go`. Today there is **no** Jackett/Torznab code anywhere in `lava-api-go` or `core/` (grep for `jackett|torznab|9117` returns nothing).
- The compose file deliberately omits a `healthcheck:` on `lava-api-go` (distroless, no `/bin/sh`); it relies on the Dockerfile `HEALTHCHECK` exec-form calling `healthprobe`, with a §6.A contract test at `lava-api-go/tests/contract/healthcheck_contract_test.go`. **Any Jackett healthcheck we add must follow the same §6.A/§6.B discipline.**

This is the architecture Jackett must slot into: a **new bridge-network service** that only `lava-api-go` talks to; the app keeps seeing one endpoint.

---

1. Jackett container image

Images & registries

- Canonical community image: **LinuxServer.io `linuxserver/jackett`**, primary pull path `lscr.io/linuxserver/jackett:latest`; also mirrored to GitHub Container Registry, GitLab Container Registry, Quay.io, and Docker Hub. (docs.linuxserver.io, hub.docker.com/r/linuxserver/jackett)
- There is **no separate official "jackett/jackett" Docker image** maintained by the Jackett project as the recommended container; LinuxServer is the de-facto community standard, and the Jackett repo itself points users to binaries/installers. (github.com/Jackett/Jackett)

Architecture support (CRITICAL for the Apple-Silicon-via-podman-VM / x86-Linux split)

LinuxServer publishes a **multi-arch manifest** — `docker pull` auto-selects the right variant. Per the official LinuxServer image doc, the architecture table is:

Architecture	Available	Per-arch tag
x86-64 (amd64)	✅	`amd64-<version tag>`
arm64 (arm64v8)	✅	`arm64v8-<version tag>`

(docs.linuxserver.io/images/docker-jackett)

→ **Both `linux/amd64` and `linux/arm64` are first-class.** This covers both Lava local-stack host profiles (Apple-Silicon macOS under the podman VM = arm64 guest; x86 Linux = amd64). The Dockerfile confirms a dedicated `Dockerfile.aarch64`. (github.com/linuxserver/docker-jackett/blob/master/Dockerfile.aarch64)

Port, base image, config layout

- Exposed port: **9117** (WebUI + API). (docs.linuxserver.io/images/docker-jackett)
- Base image: **Alpine 3.22** (as of the LinuxServer doc snapshot 2025-07-09). (docs.linuxserver.io/images/docker-jackett)
- Volumes: **`/config`** (Jackett configuration, including `ServerConfig.json` and per-indexer defs), optional `/downloads` (torrent blackhole). (docs.linuxserver.io/images/docker-jackett)
- Env vars: `PUID=1000`, `PGID=1000`, `TZ=Etc/UTC`, `AUTO_UPDATE=true` (note: auto-update is unavailable in read-only or non-root modes), `RUN_OPTS=`. (docs.linuxserver.io/images/docker-jackett)
- **Image size:** `UNCONFIRMED:` the LinuxServer doc does not state a compressed/uncompressed size. Order-of-magnitude expectation from the Alpine base + .NET runtime is "a few hundred MB"; confirm by pulling and running `podman image inspect` before relying on a number. (No primary citation for a size figure was found.)

Where the API key lives in `/config`

- Jackett generates the API key **automatically on first run** and stores it in `ServerConfig.json` (under the Jackett config dir, mapped to `/config` in the container). The key field is `api_key` in the DTO / `APIKey` in the on-disk config. Jackett does **not** rotate it afterward. (techsive.com/how-do-i-get-a-jackett-api-key, github.com/ahuacate/jackett/blob/master/ServerConfig.json, github.com/Jackett/Jackett `ServerConfig.cs`)
- `ServerConfig` fields seen in the DTO: `api_key`, `port`, `external`, `blackholedir`, `updatedisabled`, `prerelease`, `password` (admin password), `logging`, `basepathoverride`, `omdbkey`, `app_version`, `proxy_url`, `proxy_port`, `proxy_username`, `proxy_password`. (The FlareSolverr URL is **not** in this DTO — it lives in the indexer/global config, see §4.) (github.com/Jackett/Jackett `ServerConfig.cs`)

---

2. Programmatic configuration (no web UI)

The operator constraint is "configure Jackett without clicking the WebUI." Two complementary paths:

(a) API key — retrieve, don't generate

- **Preferred (automation-friendly):** let Jackett create the key on first boot, then **read `api_key` out of `/config/ServerConfig.json`** from the host (the volume is on the host). This is deterministic and avoids a chicken-and-egg.
- **Pre-seed:** you may write a chosen `APIKey` value into `ServerConfig.json` **before** first start; Jackett honors the existing value rather than generating a new one. (`UNCONFIRMED:` the exact validation/length constraints Jackett applies to a pre-seeded key — confirm by test; the key it generates is a 32-char hex-like token per community reports.) (techsive.com/how-do-i-get-a-jackett-api-key, github.com/swizzin/swizzin/issues/259)
- There is **no documented CLI flag to print/generate the API key**; Jackett's CLI flags cover `--Port`, `--ListenPublic`, etc., not key emission. So "read it from the file" is the supported automation path. (github.com/Jackett/Jackett)

(b) Indexer configuration

- Jackett stores configured indexers as JSON under the config dir (per-indexer config files in `/config`). Two automation approaches:
  1. **Write indexer JSON into the config volume** before/while the container is down, then restart. This is the most "declarative" path but the per-indexer JSON schema is **version-coupled and not a stable public contract**. `UNCONFIRMED:` Jackett does not publish a documented schema for these per-indexer files; treat them as opaque and capture by example.
  2. **Drive Jackett's own HTTP admin API** (the same endpoints the WebUI calls) — e.g. `GET /api/v2.0/indexers`, `GET /api/v2.0/indexers/<id>/config`, `POST .../config`, `POST .../api/v2.0/indexers/<id>/config` to apply credentials/FlareSolverr URL. This is how tools script Jackett headlessly. `UNCONFIRMED:` these admin endpoints are **not part of the stable Torznab surface** and are effectively internal; pin the

Jackett version and add a §6.A contract test if `lava-api-go` ever calls them.

- **§6.H secret handling:** tracker credentials (RuTracker/RuTor/IPTorrents/NNMClub) entered into Jackett indexers end up in Jackett's `/config` on the host. That volume MUST be treated as a secrets store: gitignored, never committed, never logged. The Jackett `api_key` itself is a secret — read it at runtime into `lava-api-go`'s env from the host volume; do not hardcode it (§6.R). Credentials passed to Jackett come from Lava's `.env` (already gitignored), injected into the indexer config at provisioning time.

> **Recommendation:** Use path (a)-read-from-file for the API key + path (b.2)-HTTP-admin-API for indexer setup, behind a small idempotent provisioning step in the Containers-submodule glue. Avoid hand-writing per-indexer JSON (b.1) as the primary path because the schema is undocumented.

---

3. Torznab client in Go (`lava-api-go` → Jackett)

Endpoint & query

- Results endpoint (one per indexer): `http://<jackett-host>:9117/api/v2.0/indexers/<indexer-id>/results/torznab/api`
- Capabilities: same path with `?t=caps&apikey=<key>` — returns supported search modes + categories; use this as the **readiness probe** for a given indexer (see §6).
- Aggregate endpoint: indexer-id **`all`** queries every configured indexer at once.
- Core params: `apikey` (required), `t` (`search|tvsearch|movie|music|book|caps`), `q` (free text), `cat` (comma-separated category IDs), `imdbid`, `season`, `ep`, `cache=false` (bypass Jackett cache). (deepwiki.com/Jackett/Jackett/3-torznab-api-reference, github.com/Jackett/Jackett)

Example: `GET /api/v2.0/indexers/rutracker/results/torznab/api?apikey=KEY&t=search&q=ubuntu`

Response shape (RSS/XML + torznab namespace)

Torznab returns an RSS 2.0 feed; all extended attrs live in namespace `xmlns:torznab="http://torznab.com/schemas/2015/feed"`. Each result is an `<item>` carrying standard RSS fields plus `<torznab:attr name="..." value="..."/>` entries (seeders, peers, infohash, magneturl, category, size, downloadvolumefactor, uploadvolumefactor, …) and an `<enclosure>`. (deepwiki.com/Jackett/Jackett/3-torznab-api-reference, torznab.github.io spec v1.3)

The **download link is conveyed via `<enclosure>`**, and torrents-vs-magnets use two enclosure type strings (verbatim from the Torznab torrent-support spec):

```
<!-- .torrent file -->
<enclosure url="https://yoursite.com/download.php?
         torrent=123&passkey=123456"
            length="1460985071"
            type="application/x-bittorrent" />
```

```
<!-- magnet -->
<enclosure url="magnet:?
        xt=urn:btih:c12fe1c06bba254a9dc9f519b335aa7c1367a88a&dn..."
           length="1460985071"
           type="application/x-bittorrent;x-scheme-handler/
        magnet" />
```

A magnet may **also** be supplied redundantly as `<torznab:attr name="magneturl" value="magnet:?..."/>`, and `<torznab:attr name="infohash" value="..."/>` may be present. "Multiple enclosures MAY be provided; clients SHOULD select the preferred enclosure." (torznab.github.io 1.0-Torznab-Torrent-Support)

Go libraries

- **`github.com/cardigann/cardigann/torznab`** — MIT-licensed Go package with `ResultItem`, `ResultFeed`, `Capabilities`, `Category`, `Query`, an `Indexer` interface (`Info/Search/Download/ Capabilities`), `ParseQuery(url.Values)`, and **XML marshaling** (`MarshalXML` on items/feeds/caps). Usable as both client-side parsing helpers and server-side serialization. **Caveat:** last published `v1.10.2` (2018-06-30) — it is **stale/unmaintained**; vendor a known-good revision and pin it, or lift just the struct definitions. (pkg.go.dev/github.com/cardigann/cardigann/torznab)
- **`github.com/mrobinsn/go-newznab`** — newznab/torznab XML client for Go (`newznab.New(url, key, userid, insecure)`, TV/movie search, RSS, `.nzb`/`.torrent` download). Also older; usable as a reference. A modernized fork exists at `github.com/ovrlord-app/go-newznab`. (github.com/mrobinsn/go-newznab, pkg.go.dev/github.com/ovrlord-app/go-newznab)

> **Recommendation:** Don't pull a stale module as a runtime dependency. Torznab is "RSS-over-HTTP with a known namespace" — implement a **small first-party Go decoder** in `lava-api-go` using `encoding/xml` structs (item + enclosure + `torznab:attr` slice), borrowing the field set from `cardigann/torznab` (MIT — attribution-compatible). This keeps the §6.D behavioral-coverage contract tractable and avoids inheriting an unmaintained dep. Per the Decoupled-Reusable rule, a generic Torznab client is a candidate for a `vasic-digital` submodule if a second project would want it.

The 302 → magnet edge case (must handle)

Jackett often returns a **302 redirect to a `magnet:` URI** from its download/`/dl/` proxy link, and some indexers return the download field in Jackett's **proxy-link format** rather than a raw magnet. Concretely:

- Jackett wraps non-magnet downloads (and blackhole actions) through `serverService.ConvertToProxyLink()` (the `/dl/` endpoint); **magnets are normally passed through raw and are NOT proxy-wrapped unless it's a blackhole action**. (github.com/Jackett/Jackett/issues/8252)
- When a client follows a Jackett download link, Jackett may answer **HTTP 302 whose `Location` is a `magnet:` URI**. A naive HTTP client that auto-follows redirects will try to "GET magnet:" and fail; the correct handling is to **disable redirect-following and read the `Location` header**. This is a known interop

bug class (Prowlarr's generic-torznab had to fix exactly this). (github.com/Prowlarr/Prowlarr/issues/892, github.com/qbittorrent/qBittorrent/issues/11877)
- Secondary hazards: some indexers emit **improperly URI-escaped magnet links** (e.g. 1337x), and reverse-proxy/SSL setups can produce **wrong-scheme `http://` download links**. (github.com/Jackett/Jackett/issues/8889, github.com/Jackett/Jackett/issues/1464)

> `lava-api-go`'s Torznab client MUST: (1) configure its HTTP client with `CheckRedirect: http.ErrUseLastResponse` (or equivalent) on download fetches so it captures the `Location: magnet:` instead of following it; (2) prefer the `<torznab:attr name="magneturl">` / `infohash` when present; (3) validate/normalize magnet URIs before handing them to the app. This directly serves the worklog's §2 goal ("confirm a real working download: valid `.torrent`, real non-empty `.torrent`, OR 100%-valid magnet") and the known kinozal/nnmclub null-magnet gaps.

---

4. Cloudflare / FlareSolverr (critical for IPTorrents)

Several of the operator's target private trackers (notably **IPTorrents**) sit behind Cloudflare / DDoS-Guard. Jackett's answer is **FlareSolverr**, a separate proxy container running a headless browser that solves the challenge and returns cookies/HTML.

- **Integration model:** FlareSolverr runs as its **own container** on port **8191**, exposing a JSON POST API at **`/v1`** (`request.get`, `request.post`, `sessions.create/list/destroy`). Jackett is pointed at it via a **FlareSolverr URL** in Jackett's config (e.g. `http://flaresolverr:8191`). Both Jackett and Prowlarr have built-in FlareSolverr support. (github.com/FlareSolverr/FlareSolverr, github.com/Jackett/Jackett/issues/9029, rapidseedbox.com/blog/flaresolverr-guide)
- **Architecture support:** the FlareSolverr image targets `linux/386`, `linux/amd64`, `linux/arm/v7`, and **`linux/arm64`**, with Chromium bundled in the image. (hub.docker.com/r/flaresolverr/flaresolverr, github.com/FlareSolverr/FlareSolverr)
  - `UNCONFIRMED:` FlareSolverr upstream has been in maintenance/low-activity mode and ARM Chromium has historically been the fragile path; **verify an actual arm64 boot under the podman VM on the Apple-Silicon host** before declaring it gate-ready (this matters because Lava's emulator gate already hit the "podman VM lacks /dev/kvm/HVF" class of host limitation — a Chromium-in-container boot is a separate but adjacent risk). No primary citation pins the arm64-under-podman-VM status.
- **RAM cost:** "Web browsers consume a lot of memory. If you are running FlareSolverr on a machine with few RAM, do not make many requests at once. With each request a new browser is launched." Practically this is the **heaviest** part of the stack — a Chromium per concurrent challenge. (github.com/FlareSolverr/FlareSolverr, zenrows.com/blog/flaresolverr)
- **Env / health:** key env vars `LOG_LEVEL`, `LOG_FILE`, `PORT` (8191), `HOST` (0.0.0.0), `HEADLESS` (true), `PROMETHEUS_ENABLED`. Timeout is a per-request `maxTimeout` (ms), not an env var. **No dedicated `/health` endpoint** is documented; a `POST /v1 {"cmd":"sessions.list"}` (or a trivial `request.get`) is the de-facto liveness check. License: **MIT**. (github.com/FlareSolverr/FlareSolverr)

> **Recommendation:** Include FlareSolverr, but **gate it behind a compose profile** (e.g. `cloudflare`) so it's only up when an indexer that needs it (IPTorrents) is enabled. It is the only component that materially raises the stack's RAM floor, and it's the most fragile on arm64. For trackers that don't need Cloudflare bypass (RuTracker/RuTor/NNMClub), leave it off.

---

5. Prowlarr comparison (operator chose Jackett — does Prowlarr materially ease automation?)

Dimension	Jackett	Prowlarr
Role	Torznab/Newznab **proxy**; one endpoint per indexer	Indexer **manager** that *pushes* indexers into *arr apps; also exposes Torznab/Newznab per indexer
API for automation	Torznab API (stable) + undocumented internal admin endpoints	Full **documented REST API** (`/api/v1`, with an OpenAPI spec) for indexer CRUD, search, app sync
Programmatic indexer mgmt	Per-indexer JSON in `/config` (undocumented) or internal admin calls	First-class: add/update/delete indexers, set FlareSolverr, test, search — all via REST
Indexer coverage	**Larger/older** community catalog; many niche trackers Jackett-only	Modern; some niche defs not yet ported
Stack family	standalone	Servarr (.NET/React), same base as Sonarr/Radarr
Arch	amd64 + arm64 (LinuxServer)	amd64 + arm64 (LinuxServer/Servarr)
License	**GPL-2.0**	**GPL-3.0** `UNCONFIRMED-EXACT:` Servarr family is GPL-3.0; the README badge wasn't quoted verbatim by the fetch — confirm on the repo LICENSE before relying on it

Sources: (dev.to/selfhostingsh prowlarr-vs-jackett, datahoarder.io/prowlarr-vs-jackett, github.com/Prowlarr/Prowlarr, github.com/Jackett/Jackett)

Honest assessment for *this* use case (one `lava-api-go` consuming Torznab):

- Prowlarr would **materially ease programmatic indexer management** because it has a real, documented REST API for adding/configuring/testing indexers and wiring FlareSolverr — versus Jackett's read-the-JSON-file / internal-admin-endpoint approach. If "configure indexers headlessly from `lava-api-go` or the Containers glue" is a hard requirement, Prowlarr is the lower-friction tool.
- However, Prowlarr's outward-facing **search/Torznab surface that `lava-api-go` would consume is essentially the same** Torznab-over-HTTP. The app-side integration in `lava-api-go` is nearly identical either way. Jackett's advantage is

the **broader niche-tracker catalog** (relevant for RuTracker/RuTor/NNMClub/IPTorrents) and GPL-2.0 vs GPL-3.0 distribution nuance.
- **Net:** operator's Jackett choice is sound for *consumption*; the only place Prowlarr clearly wins is *provisioning automation*. If we adopt the "read api_key from file + drive Torznab only, configure indexers once via a provisioning step" topology below, Jackett's automation gap is bounded and acceptable. Flag this as a decision point (Open Question Q1).

---

6. Resource footprint, lifecycle & health (for the §6.B "Up != healthy" contract)

RAM / CPU

- **Jackett (idle / normal):** community/issue data puts Jackett's working set at **~100 MB**, rising toward **~300 MB** after the .NET auto-update path or with many indexers; it's a .NET app with a per-indexer result **cache** (default cache TTL **2100 s / 35 min**, configurable to bound RAM). Pathological CPU spikes (e.g. "600% CPU") have been reported under specific bad-indexer conditions, so treat CPU as bounded-but-not-zero. (github.com/Jackett/Jackett/issues/4844, github.com/Jackett/Jackett/issues/4720, github.com/Jackett/Jackett/issues/12244)
  - `UNCONFIRMED:` precise idle RSS on the LinuxServer Alpine arm64 image — the figures above are from issue reports across platforms/versions, not a single authoritative benchmark. Measure on the target host and record per §6.T.2 resource discipline.
- **FlareSolverr (under search):** dominated by Chromium — **one browser process per concurrent request**; the single heaviest component. Idle is modest (a Python proxy) but each challenge spikes hundreds of MB. (github.com/FlareSolverr/FlareSolverr)

Startup time

- `UNCONFIRMED:` no primary source pins Jackett cold-start seconds; expect a few seconds for the .NET runtime + indexer-def load. FlareSolverr adds Chromium warm-up on first challenge. Measure and bake the observed value into the healthcheck `start_period`.

Health / readiness — what to probe (no native `/health`)

- **Jackett has no documented `/health` endpoint.** "It is running" is verified in practice by hitting the dashboard at `:9117`. (github.com/Jackett/Jackett)
- **§6.B-correct readiness (application-healthy, not just "Up"):** probe a **Torznab caps query** on a known indexer (or `all`): `GET http://127.0.0.1:9117/api/v2.0/indexers/all/results/torznab/api?t=caps&apikey=KEY` → HTTP 200 + a parseable `<caps>` document means Jackett is up, the API key is valid, and at least the Torznab surface is live. This is the real user-visible code path `lava-api-go` will use, so it satisfies §6.B ("end-to-end functional request, not lifecycle plumbing"). A bare TCP-9117 or a `200` on `/` would be a bluff probe.

- **FlareSolverr readiness:** `POST :8191/v1 {"cmd":"sessions.list"}` → 200.
- Add §6.A contract coverage: any compose `healthcheck:/healthprobe` invocation against Jackett must be flag-compatible with the binary it calls (same discipline as `lava-api-go/tests/contract/healthcheck_contract_test.go`).

---

7. mDNS / discovery fit

- **Jackett has no native mDNS/Bonjour advertisement.** It only binds HTTP on 9117. (Nothing in the LinuxServer or Jackett docs advertises a service type.) (docs.linuxserver.io/images/docker-jackett)
- Lava's app already discovers exactly one service type, **`_lava-api._tcp`** (= `lava-api-go`), and talks only to `lava-api-go`.

> **Recommendation — `lava-api-go` proxies; Jackett is NOT separately advertised.** Jackett stays on the internal `lava-net` bridge, reachable only by `lava-api-go` at `http://jackett:9117` (or `127.0.0.1:9117` if Jackett is also host-net). The app continues to see a single `_lava-api._tcp` endpoint; `lava-api-go` gains internal Torznab routes (e.g. a new tracker provider) that call Jackett server-side. This:
>
> - keeps the app's discovery/auth model unchanged (one endpoint, existing TLS + HMAC auth),
> - keeps the Jackett `api_key` and tracker credentials **server-side only** (never shipped to the device — §6.H),
> - matches the Decoupled-Reusable boundary (Jackett = third-party capability; `lava-api-go` = Lava glue/domain). Advertising Jackett separately on the LAN would expose an unauthenticated 9117 admin surface to the network — **reject that option** on security grounds.

---

8. Licensing for distribution

- **Jackett: GPL-2.0.** Confirmed via the repo's license badge. (github.com/Jackett/Jackett)
- **FlareSolverr: MIT.** (github.com/FlareSolverr/FlareSolverr)
- **Prowlarr (if ever chosen): GPL-3.0** (`UNCONFIRMED-EXACT:` Servarr family is GPL-3.0; verify the LICENSE file before relying on it). (github.com/Prowlarr/Prowlarr)
- **`cardigann/torznab` Go package: MIT** (attribution-compatible if we lift struct definitions). (pkg.go.dev/github.com/cardigann/cardigann/torznab)

What GPL-2.0 means when we ship Jackett as a container in a user-run local stack:

- **We do not modify Jackett** — we run an unmodified upstream/LinuxServer image. Distributing an **unmodified GPL-2.0 binary** (as a referenced/pulled container image) triggers the GPL "convey" obligations: when we distribute the image/compose, we must **either accompany it with the corresponding source or a written offer of source**, and **preserve copyright/license notices**. Because

we pull a published upstream image rather than re-hosting a modified one, the practical obligation is satisfied by: (a) **not stripping** Jackett's notices/license, and (b) **pointing to the upstream source** (`github.com/Jackett/Jackett`, tag-pinned) in our distribution docs/NOTICE. `UNCONFIRMED-LEGAL:` this is an engineering reading of GPL-2.0 §3, not legal advice — if Lava **redistributes the image bytes** (e.g. re-pushes Jackett to Lava's own registry) vs. **merely references `lscr.io/...`** changes the offer-of-source burden; have the operator confirm the distribution mode. (github.com/Jackett/Jackett (GPL-2.0))

- **Compose/orchestration is "mere aggregation."** Our docker-compose + `lava-api-go` calling Jackett over HTTP/Torznab is an **arms-length aggregation**, not a derivative work — GPL-2.0 does not reach `lava-api-go` through an HTTP boundary. So Lava's own (proprietary-or-otherwise) code is **not** forced under GPL by talking to Jackett over the network. (`UNCONFIRMED-LEGAL:` standard FOSS interpretation; not legal advice.)
- **Pin the Jackett image by digest** (not `:latest`) for reproducibility + a stable source reference, consistent with the existing compose policy of pinning observability images.

Recommended local-stack topology

A new compose service set, fronted by `lava-api-go`, on the existing bridge. App-facing surface is unchanged (still one `_lava-api._tcp` endpoint).

```
Android app ──mDNS(_lava-api._tcp)──▶ lava-api-go (:8443, host-
net, TLS+HMAC)
                                          │  (server-side HTTP,
holds api_key)
                                          ▼
                                    jackett (:9117)  ── on
lava-net bridge, NOT advertised
                                          │
                              (only when needed,
profile=cloudflare)
                                          ▼
                                flaresolverr (:8191)  ── on
lava-net bridge
```

Services (sketch — not committed)

Service	Image (pin by digest)	Network	Port	Profile	Notes
`lava-jackett`	`lscr.io/linuxserver/jackett`	`lava-net` (bridge)	9117 (internal only)	`jackett`	`/config` = gitignored host volume holding `ServerConfig.json` + indexer creds
`lava-flaresolverr`	`flaresolverr/flaresolverr`	`lava-net`	8191 (internal only)	`cloudflare`	only up when IPTorrents (or other CF tracker) enabled

Service	Image (pin by digest)	Network	Port	Profile	Notes
`lava-api-go`	(existing)	host-net	8443	`api-go`	gains internal Torznab client + new provider; reads Jackett `api_key` from host volume at start

Who holds the apikey

- **Jackett generates it** → persisted in the gitignored `/config/ServerConfig.json` host volume.
- **`lava-api-go` reads it at startup** from that host path (or an env var populated from it by the Containers glue) — **never** hardcoded (§6.R), **never** sent to the device (§6.H).

Healthchecks (§6.B-correct)

- Jackett: `t=caps` Torznab probe (200 + parseable `<caps>`), `start_period` ≥ measured cold-start.
- FlareSolverr: `POST /v1 {"cmd":"sessions.list"}` → 200.
- Add §6.A contract test for any healthprobe flags.

FlareSolverr: yes, but profiled. Include it for IPTorrents; keep it behind a `cloudflare`

profile so the RAM-heavy Chromium isn't running for RuTracker/RuTor/NNMClub-only sessions.

Ports exposed to LAN

- **None** for Jackett/FlareSolverr. Both stay internal to `lava-net`; only `lava-api-go:8443` is LAN-visible (security: no unauthenticated 9117 admin surface on the network).

Torznab client

- First-party `encoding/xml` decoder in `lava-api-go` (struct field set borrowed from MIT `cardigann/torznab`); **disable redirect-following on download fetches** to capture `Location: magnet:`; prefer `torznab:attr magneturl/infohash`; validate magnet URIs. Candidate for a `vasic-digital` submodule if reused.

OPEN QUESTIONS / UNCONFIRMED

Open questions (need an operator/architecture decision):

- **Q1 — Jackett vs Prowlarr final call.** Operator chose Jackett; Prowlarr only wins on *provisioning automation* (documented REST indexer CRUD). Accept Jackett's read-file + provision-once model, or switch to Prowlarr for headless indexer management? (Consumption side is ~equal.)
- **Q2 — Distribution mode for the Jackett image.** Do we *reference* `lscr.io/...` by digest (lighter GPL offer-of-source burden) or *re-push* Jackett into a Lava-owned registry (heavier burden)? Drives §8 obligations.
- **Q3 — Indexer provisioning path.** Drive Jackett's internal admin HTTP endpoints (version-coupled, undocumented) vs. write per-indexer `/config` JSON (undocumented schema)? Either needs a §6.A contract test + pinned Jackett version.
- **Q4 — Jackett network mode.** Put Jackett on `lava-net` bridge (clean isolation; `lava-api-go` must reach it across host-net?bridge — use `host.containers.internal:9117` like the Prometheus scrape already does) vs. host-net (simpler `127.0.0.1:9117` but more exposed). Bridge is recommended.
- **Q5 — FlareSolverr on arm64-under-podman-VM.** Must be boot-verified on the Apple-Silicon host before it's gate-eligible (§6.AH container/VM-only emulation mandate spirit applies to any browser-in-container too).

UNCONFIRMED facts to verify by direct test before implementation:

- `UNCONFIRMED:` Jackett LinuxServer image size (no published figure).
- `UNCONFIRMED:` Jackett idle RSS on the target arm64 Alpine image (issue reports span versions/OSes).
- `UNCONFIRMED:` Jackett cold-start seconds (set healthcheck `start_period` from measurement).
- `UNCONFIRMED:` exact validation rules for a *pre-seeded* `api_key` value.
- `UNCONFIRMED:` stability of Jackett's internal admin/indexer-config HTTP endpoints across versions (treat as internal; pin + contract-test).
- `UNCONFIRMED:` per-indexer `/config` JSON schema (undocumented; capture by example).
- `UNCONFIRMED:` FlareSolverr arm64 Chromium boot under the podman VM on macOS (README claims arm64 support; project is low-activity — verify live).
- `UNCONFIRMED-EXACT:` Prowlarr LICENSE is GPL-3.0 (Servarr family is; confirm the file).
- `UNCONFIRMED-LEGAL:` §8 GPL-2.0 convey/offer-of-source reading and "mere aggregation" of `lava-api-go`?Jackett over HTTP — engineering interpretation, not legal advice.

Source index (all external claims)

- LinuxServer Jackett image doc — https://docs.linuxserver.io/images/docker-jackett/
- LinuxServer Jackett on Docker Hub — https://hub.docker.com/r/linuxserver/jackett

- LinuxServer aarch64 Dockerfile — https://github.com/linuxserver/docker-jackett/blob/master/Dockerfile.aarch64
- Jackett repo (license GPL-2.0, CLI, default port) — https://github.com/Jackett/Jackett
- Jackett `ServerConfig.cs` DTO (api_key/proxy fields) — https://github.com/Jackett/Jackett/blob/570ea5bb51b6c62a0f952a4f08e86abc8bfc3503/src/Jackett/Models/DTO/ServerConfig.cs
- Jackett API key location (ServerConfig.json) — https://techsive.com/how-do-i-get-a-jackett-api-key/ ; https://github.com/ahuacate/jackett/blob/master/ServerConfig.json ; https://github.com/swizzin/swizzin/issues/259
- Torznab API reference (endpoint, params, namespace) — https://deepwiki.com/Jackett/Jackett/3-torznab-api-reference
- Torznab spec v1.3 (extended attrs) — https://torznab.github.io/spec-1.3-draft/torznab/Specification-v1.3.html
- Torznab torrent-support (enclosure/magnet XML) — https://torznab.github.io/spec-1.3-draft/revisions/1.0-Torznab-Torrent-Support.html
- 302→magnet / proxy link — https://github.com/Prowlarr/Prowlarr/issues/892 ; https://github.com/Jackett/Jackett/issues/8252 ; https://github.com/qbittorrent/qBittorrent/issues/11877 ; https://github.com/Jackett/Jackett/issues/8889 ; https://github.com/Jackett/Jackett/issues/1464
- Go torznab libs — https://pkg.go.dev/github.com/cardigann/cardigann/torznab ; https://github.com/mrobinsn/go-newznab ; https://pkg.go.dev/github.com/ovrlord-app/go-newznab
- FlareSolverr (arch, MIT, API, RAM) — https://github.com/FlareSolverr/FlareSolverr ; https://hub.docker.com/r/flaresolverr/flaresolverr ; https://www.rapidseedbox.com/blog/flaresolverr-guide ; https://github.com/Jackett/Jackett/issues/9029 ; https://www.zenrows.com/blog/flaresolverr
- Jackett memory/CPU — https://github.com/Jackett/Jackett/issues/4844 ; https://github.com/Jackett/Jackett/issues/4720 ; https://github.com/Jackett/Jackett/issues/12244
- Prowlarr — https://github.com/prowlarr/prowlarr ; https://dev.to/selfhostingsh/prowlarr-vs-jackett-which-indexer-manager-al4 ; https://datahoarder.io/prowlarr-vs-jackett/ ; https://wiki.servarr.com/prowlarr/quick-start-guide